

T19 — Documentazione con Javadoc e Unit Testing con JUnit 5

Convenzioni Javadoc, Architettura JUnit 5, Annotazioni di Ciclo di Vita ed Asserzioni

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

0.1 1. Analisi Teorica Approfondita (Slide T19)

La lezione 19 approfondisce due pratiche ingegneristiche irrinunciabili nello sviluppo professionale in Java:

1. **Documentazione automatizzata del codice** tramite lo standard **Javadoc**.
2. **Collaudo e Unit Testing automatizzato** tramite il framework **JUnit 5 (Jupiter)** e il plugin Maven **Surefire**.

0.1.1 1.1 Documentazione con Javadoc (Slide 3-11)

La documentazione del codice sorgente è essenziale per manutenibilità, collaborazione e pubblicazione di API riusabili. In Java la documentazione vive a stretto contatto con il codice nei cosiddetti **Doc Comments**:

- * **Sintassi:** Delimitati da `/**` all’inizio e `*/` alla fine. Ogni riga intermedia inizia convenzionalmente con `*`: `java /** * Descrizione sintetica del componente. * <p>Supporta l'uso di tag HTML come paragrafi, link e formattazione.</p> */`
- * **I Tag Javadoc Standard (Slide 5-6):** Iniziano con `@`, sono *case-sensitive* e devono comparire a inizio riga:
 - * `@param <nome>`: descrive un parametro formale di un metodo o costruttore.
 - * `@return`: descrive il significato del valore restituito da un metodo (omesso nei metodi `void` e nei costruttori).
 - * `@throws <Eccezione>` (o `@exception`): descrive le condizioni anomale che causano il sollevamento di una determinata eccezione.
 - * `{@link <package.Classe#metodo>}`: genera un ipertesto cliccabile verso un’altra classe o metodo.
 - * `@author`: autore del modulo.
 - * `@since <versione>`: versione a partire dalla quale la feature è disponibile.
 - * `@version`: versione corrente del sorgente.
 - * `@deprecated`: avvisa che l’elemento è obsoleto, illustrandone il motivo e l’alternativa moderna da utilizzare.

Regole di Visibilità e Generazione (Slide 9-11)

- **Default di Javadoc:** Per impostazione predefinita, Javadoc genera la documentazione per le sole entità **public** e **protected** (l’interfaccia pubblica esposta ai client). I campi e metodi **private** o **package-private** vengono omessi.
- **Inclusione del privato:** Si deve passare esplicitamente il flag `-private` da riga di comando:

```
1 javadoc -private -d doc *
```

- **Integrazione Maven (maven-javadoc-plugin, Slide 11):** Aggiungendo il plugin nel `pom.xml`, la documentazione viene generata con un singolo comando standard:

```
1 mvn javadoc:javadoc
```

0.1.2 1.2 Unit Testing con JUnit (Slide 13-16)

Il testing programmatico garantisce la correttezza delle singole unità software in isolamento (metodi o singole classi):

- * **Vantaggi sistemici:**
- * **Test-Driven Development (TDD):** scrivere i test prima ancora di implementare il codice applicativo.
- * **Regression Testing:** certezza che refactoring o nuove feature non rompano comportamenti pregressi funzionanti.
- * Riduzione drastica del tempo speso nel debugging manuale.
- * **Anatomia di un Test Case:** 1. Preparazione dell'input (*Arrange / Setup*). 2. Esecuzione del metodo sotto test (*Act*). 3. Confronto dell'output effettivo (*Actual*) con l'output atteso (*Expected*) tramite **asserzioni** (*Assert*).
- * **Mecanismo di Fallimento:**
- * Un metodo di test restituisce sempre `void`.
- * Se tutte le asserzioni sono soddisfatte, il metodo termina normalmente e JUnit lo contrassegna come **PASSED** (verde).
- * Se un'asserzione fallisce, solleva un errore di tipo `AssertionError` (o `AssertionFailedError`). Il framework JUnit cattura l'errore, contrassegna il test come **FAILED** (rosso) con il report della discrepanza, e **prosegue regolarmente con i test successivi** senza arrestare l'intera suite.

0.1.3 1.3 Asserzioni in JUnit 5 (org.junit.jupiter.api.Assertions, Slide 17-21)

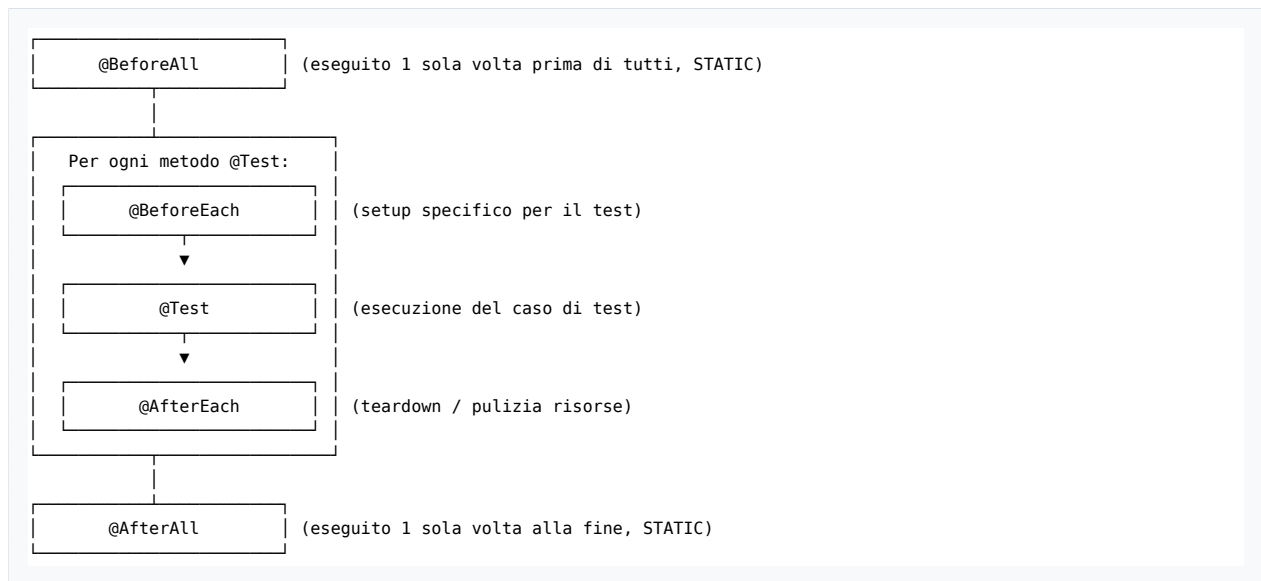
Il pacchetto Jupiter standardizza una vasta famiglia di metodi statici:

Metodo Asserzione	Comportamento
<code>assertEquals(expected, actual, [msg])</code>	Verifica che <code>expected.equals(actual)</code> . Valido per primitivi e oggetti.
<code>assertTrue(condition, [msg])</code>	Verifica che la condizione booleana sia <code>true</code> .
<code>assertFalse(condition, [msg])</code>	Verifica che la condizione booleana sia <code>false</code> .
<code>assertArrayEquals(expected, actual)</code>	Confronta il contenuto e l'ordinamento di due array elemento per elemento.
<code>assertNull(obj) / assertNotNull(obj)</code>	Verifica se il puntatore è <code>null</code> o non nullo.
<code>assertThrows(Exception.class, executable)</code>	Verifica che l'esecuzione di una lambda/blocco sollevi tassativamente l'eccezione attesa!

Best Practice: Static Imports (Slide 21) Per evitare di anteporre continuamente `Assertions.` davanti a ogni chiamata, si usa l'import statico:

```
1 import static org.junit.jupiter.api.Assertions.assertEquals;
2 import static org.junit.jupiter.api.Assertions.assertThrows;
```

0.1.4 1.4 Il Ciclo di Vita del Test e le Annotazioni JUnit 5 (Slide 22)



- @BeforeAll: Inizializzazione “pesante” o condivisa tra tutti i test (es. apertura connessione, setup di un database o di costanti immutabili). **Deve essere static.**
- @AfterAll: Chiusura finale delle risorse globali. **Deve essere static.**
- @BeforeEach: Inizializzazione fresca prima di *ciascun* test per garantire l’isolamento e l’indipendenza dei test (evitando che un test modifichi lo stato di un altro).
- @AfterEach: Pulizia post-test (es. cancellazione di file temporanei creati durante il test).
- @DisplayName("Descrizione chiara"): Personalizza il nome del test visualizzato nei report o nell’IDE.
- @Disabled: Disabilita temporaneamente il test senza doverlo commentare o cancellare.

0.1.5 1.5 Organizzazione del Progetto e Convenzioni Maven (Slide 23-24)

Maven e i moderni build tool impongono una struttura standard di cartelle: * **src/main/java**: Contiene il codice sorgente dell’applicazione (es. `it.oop.core.Date`). * **src/test/java**: Contiene esclusivamente le classi di collaudo. * **Allineamento dei Package (Regola Fondamentale)**: Una classe di test che verifica `it.oop.core.Date` **deve risiedere nello stesso identico package `it.oop.core`** (all’interno di `src/test/java`). In questo modo la classe di test ha accesso non solo ai membri `public`, ma anche a tutti i metodi e campi con visibilità di **package (package-private)**, facilitando il testing interno senza forzare l’apertura a `public` di dettagli architetturali riservati!

0.2 2. Analisi Dettagliata del Codice (Lezione18/MavenDate)

In Lezione18/MavenDate troviamo la configurazione completa del plugin Maven Surefire e due classi di test che collaudano la gerarchia di `Date`.

0.2.1 2.1 Configurazione in pom.xml

```
1 <build>
2   <plugins>
3     <!-- Plugin Javadoc (Slide 11) -->
4     <plugin>
5       <groupId>org.apache.maven.plugins</groupId>
6       <artifactId>maven-javadoc-plugin</artifactId>
```

```
7         <version>3.6.2</version>
8         <configuration>
9             <source>1.8</source>
10            <show>private</show> <!-- Include metodi e campi privati -->
11        </configuration>
12    </plugin>
13</plugins>
14</build>
15
16<dependencies>
17    <!-- Motore di esecuzione JUnit 5 (Slide 24) -->
18    <dependency>
19        <groupId>org.junit.jupiter</groupId>
20        <artifactId>junit-jupiter-engine</artifactId>
21        <version>5.10.0</version>
22        <scope>test</scope> <!-- Visibile solo durante la fase di test -->
23    </dependency>
24    <!-- Maven Surefire Plugin per l'esecuzione di 'mvn test' -->
25    <dependency>
26        <groupId>org.apache.maven.plugins</groupId>
27        <artifactId>maven-surefire-plugin</artifactId>
28        <version>3.5.4</version>
29    </dependency>
30</dependencies>
```

0.2.2 2.2 TestDate.java

Questa classe collauda sia il flusso nominale (costruzione valida) che il flusso eccezionale (lancio di eccezioni):

```
1 package it.oop.core;
2
3 import it.oop.exception.IllegalDateException;
4 import org.junit.jupiter.api.Assertions;
5 import org.junit.jupiter.api.Test;
6 import static org.junit.jupiter.api.Assertions.assertThrows;
7
8 public class TestDate {
9
10     private Date date;
11
12     // 1. Collaudo del caso nominale con assertEquals
13     @Test
14     public void testDateConstructor() {
15         date = new Date(1, 12, 2025);
16         Assertions.assertEquals(1, date.getDay());
17         Assertions.assertEquals(12, date.getMonth());
18         Assertions.assertEquals(2025, date.getYear());
19     }
20
21     // 2. Collaudo del caso eccezionale con assertThrows e Lambda
22     @Test
23     public void testDateConstructorException() {
24         // Verifica che passando un anno negativo (-2), il costruttore sollevi tassativamente
25         // IllegalDateException
26         assertThrows(IllegalDateException.class, () -> date = new Date(1, 12, -2));
27     }
28 }
```

0.2.3 2.3 TestItalianDate.java

Esemplifica l'uso della fixture `@BeforeAll` e mette in luce il superamento della vecchia parola chiave `assert`:

```
1 package it.oop.core;
2
3 import org.junit.jupiter.api.Assertions;
4 import org.junit.jupiter.api.BeforeAll;
5 import org.junit.jupiter.api.Test;
6
7 public class TestItalianDate {
8     private static ItalianDate date;
9
10    // Fixture statica eseguita una volta sola prima di tutti i test
11    @BeforeAll
12    public static void setup() {
13        date = new ItalianDate(1, 1, 1970);
14    }
15
16    @Test
17    public void printFormatTest() {
18        // Verifica del formato con JUnit 5:
19        Assertions.assertEquals("dd/mm/yyyy", date.printFormat());
20
21        // Confronto con la vecchia sintassi 'assert' del linguaggio (commentata dal docente):
22        // assert date.printFormat().equals("dd/mm/yyyy") : "wrong format";
23    }
24 }
```

0.3 3. Risultato di Esecuzione Reale da Terminale

Comando eseguito nel progetto [Lezione18/MavenDate](#):

```
1 mvn test
```

Output ottenuto:

```
1 [INFO] -----
2 [INFO]  T E S T S
3 [INFO] -----
4 [INFO] Running it.oop.core.TestDate
5 [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.106 s -- in
   it.oop.core.TestDate
6 [INFO] Running it.oop.core.TestItalianDate
7 [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.021 s -- in
   it.oop.core.TestItalianDate
8 [INFO]
9 [INFO] Results:
10 [INFO]
11 [INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
12 [INFO]
13 [INFO] -----
14 [INFO] BUILD SUCCESS
15 [INFO] -----
```

I 3 test sono stati eseguiti con successo, validando in modo formale la suite del progetto e generando i relativi file di report XML e TXT nella cartella `target/surefire-reports/`.

Dimmi “vai” per procedere con il **Blocco 19** (Lezione 19 / Slide T20 — *Java I/O*, stream di byte vs caratteri, `InputStream`, `OutputStream`, `Reader`, `Writer`, `BufferedReader`, serializzazione, gestione delle risorse con `try-with-resources` e analisi di `MainIO.java` in [Lezione19/MavenDate](#)).